

Sets

1. What are Sets ?

- Sets are an unordered
- unchangeable (elements contained must be immutable)
- unique collection of data items.
- Sets do not maintain insertion order, do not assign index values, and automatically eliminate duplicate values.
- Sets are heterogeneous and can store combinations of different immutable data types (integers, strings, floats, booleans).

1.1 Creating a Set Using Curly Brackets

The simplest way to create a set is by placing comma-separated data items inside curly brackets {}.

Syntax

```
Python
set_name = {item1, item2, item3}
```

Example

```
Python
sample_set = {'Mark', 'Jessa', 25, 75.25}
print(sample_set)
```

Output

```
{25, 'Mark', 75.25, 'Jessa'}
```

1.2 Creating a Set Using the set() Constructor

We can create a set by passing an iterable object (such as a list, tuple, or range) into the built-in set() constructor.

Syntax

```
Python
set_name = set(iterable)
```

Example

```
Python
book_set = set(('Harry Potter', 'Angels and Demons', 'Atlas
Shrugged'))
print(book_set)
print(type(book_set))
```

Output

```
{'Harry Potter', 'Atlas Shrugged', 'Angels and Demons'}
<class 'set'>
```

1.3 Creating a Set from a List to Remove Duplicates

Passing a list containing duplicate values into set() creates a set with all duplicate entries automatically removed.

Syntax

```
Python
set_name = set(list_name)
```

Example

```
Python
number_list = [20, 30, 20, 30, 50, 30]
sample_set = set(number_list)
print(sample_set)
```

Output

```
{50, 20, 30}
```

1.4 Attempting to Create a Set with Mutable Elements

A set itself is mutable, but all contained elements must be immutable. Trying to add mutable objects like lists or dictionaries raises a `TypeError`.

Syntax

```
Python
# Invalid: set containing a mutable list
set_name = {item1, [element1, element2]}
```

Example

```
Python
# Attempting to include a list in a set raises an error
sample_set = {'Mark', 'Jessa', [35, 78, 92]}
print(sample_set)
```

Output

```
TypeError: unhashable type: 'list'
```

1.5 Creating an Empty Set

To create an empty set, we must use `set()`. Using empty curly brackets `{}` creates an empty dictionary (class `'dict'`).

Syntax

```
Python  
empty_set = set()
```

Example

```
Python  
empty_set = set()  
empty_dict = {}  
print(type(empty_set))  
print(type(empty_dict))
```

Output

```
<class 'set'>  
<class 'dict'>
```

2. Accessing Items and Finding Length

2.1 Accessing Set Items Using a for Loop

Since sets are unordered and have no index numbers, we cannot access items using `set[index]`. We access items by iterating through the set using a for loop.

Syntax

Python

```
for item in set_name:  
    # Code to execute
```

Example

Python

```
book_set = {'Harry Potter', 'Angels and Demons', 'Atlas  
Shrugged'}  
for book in book_set:  
    print(book)
```

Output

```
Angels and Demons  
Atlas Shrugged  
Harry Potter
```

2.2 Checking if an Item Exists

The `in` operator checks whether an item exists in a set and returns `True` or `False`.

Syntax

Python

```
item in set_name
```

Example

Python

```
book_set = {'Harry Potter', 'Angels and Demons', 'Atlas  
Shrugged'}  
print('Harry Potter' in book_set)  
print('A Man called Ove' in book_set)
```

Output

True

False

2.3 Finding the Length of a Set

The len() function returns the total number of unique elements present in a set.

Syntax

Python

```
len(set_name)
```

Example

Python

```
book_set = {'Harry Potter', 'Angels and Demons', 'Atlas  
Shrugged'}  
print(len(book_set))
```

Output

3. Adding Items to a Set

3.1 Adding a Single Item with add()

The add() method adds a single element to the set.

Syntax

```
Python  
set_name.add(element)
```

Example

```
Python  
book_set = {'Harry Potter', 'Angels and Demons'}  
book_set.add('The God of Small Things')  
print(book_set)
```

Output

```
{'Harry Potter', 'The God of Small Things', 'Angels and Demons'}
```

3.2 Adding Multiple Items with update()

The update() method adds multiple items from an iterable (such as a list or tuple) to the set.

Syntax

Python

```
set_name.update(iterable)
```

Example

Python

```
book_set = {'Harry Potter', 'Angels and Demons'}  
book_set.update(['Atlas Shrugged', 'Ulysses'])  
print(book_set)
```

Output

```
{'The God of Small Things', 'Angels and Demons', 'Atlas Shrugged',  
'Harry Potter', 'Ulysses'}
```

4. Removing Items from a Set

4.1 Removing an Item with remove()

The `remove()` method deletes a specific item from the set. If the item does not exist, it raises a `KeyError`.

Syntax

Python

```
set_name.remove(item)
```

Example

Python

```
color_set = {'red', 'orange', 'yellow', 'white', 'black'}  
color_set.remove('yellow')  
print(color_set)
```

Output

```
{'red', 'orange', 'white', 'black'}
```

4.2 Removing an Item with discard()

- The discard() method deletes a specific item from the set if it exists.
- Unlike remove(), it does not raise an error if the item is missing.

Syntax

Python

```
set_name.discard(item)
```

Example

Python

```
color_set = {'red', 'orange', 'white', 'black'}  
color_set.discard('yellow')  
print(color_set)
```

Output

```
{'red', 'orange', 'white', 'black'}
```

4.3 Removing an Item with pop()

The pop() method removes and returns an arbitrary, random element from the set.

Syntax

```
Python  
removed_item = set_name.pop()
```

Example

```
Python  
color_set = {'red', 'orange', 'yellow'}  
deleted_item = color_set.pop()  
print(deleted_item)
```

Output

```
red
```

4.4 Clearing All Items with clear()

The clear() method removes all elements from the set, leaving an empty set set().

Syntax

```
Python  
set_name.clear()
```

Example

Python

```
color_set = {'red', 'orange', 'yellow'}  
color_set.clear()  
print(color_set)
```

Output

```
set()
```

4.5 Deleting a Set with del

The del keyword completely deletes the set variable from memory.

Syntax

Python

```
del set_name
```

Example

Python

```
color_set = {'red', 'orange', 'yellow'}  
del color_set
```

Output

```
(No output, set deleted from memory)
```

5. Mathematical Set Operations

5.1 Union of Sets

- Returns a new set containing all unique items from both sets.
- Can be performed using the | operator or union() method.

Syntax

Python

```
set3 = set1 | set2  
set3 = set1.union(set2)
```

Example

Python

```
color_set = {'violet', 'indigo', 'blue', 'green', 'yellow'}  
remaining_colors = {'indigo', 'orange', 'red'}  
vibgyor_colors = color_set | remaining_colors  
print(vibgyor_colors)
```

Output

```
{'indigo', 'blue', 'violet', 'yellow', 'red', 'orange', 'green'}
```

5.2 Intersection of Sets

Returns a new set containing only elements common to both sets using the & operator or intersection() method.

Syntax

Python

```
set3 = set1 & set2  
set3 = set1.intersection(set2)
```

Example

Python

```
color_set = {'violet', 'indigo', 'blue', 'green', 'yellow'}  
remaining_colors = {'indigo', 'orange', 'red'}  
new_set = color_set & remaining_colors  
print(new_set)
```

Output

```
{'indigo'}
```

5.3 Intersection Update

The `intersection_update()` method modifies the original set in-place to keep only common elements, returning `None`.

Syntax

Python

```
set1.intersection_update(set2)
```

Example

Python

```
color_set = {'violet', 'indigo', 'blue', 'green', 'yellow'}
remaining_colors = {'indigo', 'orange', 'red'}
color_set.intersection_update(remaining_colors)
print(color_set)
```

Output

```
{'indigo'}
```

5.4 Difference of Sets

Returns elements present only in the first set using the - operator or difference() method.

Syntax

Python

```
set3 = set1 - set2
set3 = set1.difference(set2)
```

Example

Python

```
color_set = {'violet', 'indigo', 'blue', 'green', 'yellow'}
remaining_colors = {'indigo', 'orange', 'red'}
print(color_set - remaining_colors)
```

Output

```
{'violet', 'blue', 'green', 'yellow'}
```

5.5 Difference Update

The `difference_update()` method removes all elements of another set from the calling set in-place.

Syntax

```
Python  
set1.difference_update(set2)
```

Example

```
Python  
color_set = {'violet', 'indigo', 'blue', 'green', 'yellow'}  
remaining_colors = {'indigo', 'orange', 'red'}  
color_set.difference_update(remaining_colors)  
print(color_set)
```

Output

```
{'violet', 'blue', 'green', 'yellow'}
```

5.6 Symmetric Difference of Sets

Returns elements that are in either set, but not in both, using the ^ operator or symmetric_difference() method.

Syntax

Python

```
set3 = set1 ^ set2  
set3 = set1.symmetric_difference(set2)
```

Example

Python

```
color_set = {'violet', 'indigo', 'blue', 'green', 'yellow'}  
remaining_colors = {'indigo', 'orange', 'red'}  
unique_items = color_set ^ remaining_colors  
print(unique_items)
```

Output

```
{'blue', 'orange', 'violet', 'green', 'yellow', 'red'}
```

5.7 Symmetric Difference Update

Updates the calling set in-place with the symmetric difference of both sets.

Syntax

Python

```
set1.symmetric_difference_update(set2)
```

Example

Python

```
color_set = {'violet', 'indigo', 'blue', 'green', 'yellow'}
remaining_colors = {'indigo', 'orange', 'red'}
color_set.symmetric_difference_update(remaining_colors)
print(color_set)
```

Output

```
{'yellow', 'green', 'red', 'blue', 'orange', 'violet'}
```

6. Copying, Relationships, and Sorting

6.1 Copying a Set

- Copying can be done using `copy()`, `set()`, or the `=` operator.
- The `=` operator creates a reference copy (changes affect both), while `copy()` creates a new independent copy.

Syntax

Python

```
set2 = set1.copy()
set3 = set(set1)
set4 = set1 # Reference copy
```

Example

Python

```
color_set = {'violet', 'blue', 'green', 'yellow'}
color_set2 = color_set.copy()
color_set3 = set(color_set)
color_set4 = color_set
print(color_set2)
print(color_set3)
print(color_set4)
```

Output

```
{'green', 'yellow', 'blue', 'violet'}, {'green', 'yellow', 'blue', 'violet'}, {'green', 'yellow', 'blue', 'violet'}
```

6.2 Subset and Superset

- `issubset()` checks if all elements of a set are in another set.
- `issuperset()` checks if a set contains all elements of another set.

Syntax

Python

```
set2.issubset(set1)
```

```
set1.issuperset(set2)
```

Example

Python

```
color_set1 = {'violet', 'indigo', 'blue', 'green', 'yellow',  
             'orange', 'red'}  
color_set2 = {'indigo', 'orange', 'red'}  
print(color_set2.issubset(color_set1))  
print(color_set1.issuperset(color_set2))
```

Output

True

True

6.3 Disjoint Sets

The `isdisjoint()` method returns `True` if two sets have no common elements.

Syntax

Python

```
set1.isdisjoint(set2)
```

Example

Python

```
color_set1 = {'violet', 'blue', 'yellow', 'red'}
```

```
color_set2 = {'orange', 'green'}  
print(color_set2.isdisjoint(color_set1))
```

Output

```
True
```

6.4 Sorting a Set

Since sets are unordered, sorting a set with `sorted()` returns a sorted list, which can then be converted back into a set if desired.

Syntax

```
Python  
sorted_list = sorted(set_name)
```

Example

```
Python  
set1 = {20, 4, 6, 10, 8, 15}  
sorted_list = sorted(set1)  
print(sorted_list)
```

Output

```
[4, 6, 8, 10, 15, 20]
```

7. Built-in Functions and Advanced Set Types

7.1 The all() and any() Functions

- all() returns True if all elements in the set are truthy.
- any() returns True if at least one element is truthy.

Syntax

```
Python
all(set_name)
any(set_name)
```

Example

```
Python
set1 = {1, 2, 3, 4}
set2 = {0, 2, 4, 6, 8}
print(all(set1))
print(all(set2))
print(any(set2))
```

Output

```
True
False
True
```

7.2 The max() and min() Functions

max() returns the largest element and min() returns the smallest element in a set.

Syntax

Python

```
max(set_name)
```

```
min(set_name)
```

Example

Python

```
set1 = {2, 4, 6, 10, 8, 15}
```

```
set2 = {'ABC', 'abc'}
```

```
print(max(set1))
```

```
print(min(set1))
```

```
print(max(set2))
```

Output

```
15
```

```
2
```

```
abc
```

7.3 Frozen Set (frozenset)

- A frozenset is an immutable, read-only set created using `frozenset()`.
- It supports mathematical operations (union, intersection), but prohibits mutating methods like `add()` or `remove()`.

Syntax

Python

```
f_set = frozenset(iterable)
```

Example

Python

```
rainbow = ('violet', 'indigo', 'blue', 'green', 'yellow',  
           'orange', 'red')  
f_set = frozenset(rainbow)  
print(f_set)
```

Output

```
frozenset({'green', 'yellow', 'indigo', 'red', 'blue', 'violet',  
           'orange'})
```

7.4 Nested Sets Using frozenset

- Standard sets cannot be nested inside another set because sets are mutable.
- To create nested sets, we place frozenset objects inside an outer set.

Syntax

Python

```
nested_set = {frozenset(iterable1), frozenset(iterable2)}
```

Example

Python

```
rainbow = ('violet', 'indigo', 'blue')
other_colors = ('white', 'black', 'pink')
nested_set = {frozenset(rainbow), frozenset(other_colors)}
print(len(nested_set))
```

Output

2

7.5 Set Comprehension

Set comprehension offers a concise syntax to create sets using iteration and optional conditions in a single line.

Syntax

Python

```
output_set = {expression for variable in input_set if condition}
```

Example

Python

```
square_set = {var ** 2 for var in range(1, 10) if var % 2 == 0}
print(square_set)
```

Output

```
{16, 64, 4, 36}
```